RELATÓRIO DE TUNNING DE BANCO DE DADOS - AP1

Aluno: Matheus Diirr Aguiar Veiga de Souza
Disciplina: Data Base Tunning
Instituição: Centro Universitário Salesiano (UniSales)

Parte 01 e Linha de Base

1. Caracterização do Ambiente

- **SGBD:** PostgreSQL 15 (executado via contêiner Docker isolado).
- **Base de Dados Objeto de Estudo:** AdventureWorks (Cenário OLTP).
- **Volume e Modelo de Dados:** 68 tabelas ativas distribuídas em múltiplos esquemas organizacionais (`sales` , `production` , `person` , `purchasing` , `humanresources`).
- **Hardware de Experimentação:** Processador AMD Ryzen 5 5600G, 16 GB de RAM, SSD NVMe 2.0 e GPU RTX 4060.

Nota de Contexto: Embora o hardware local possua alta taxa de transferência (I/O) e processamento rápido, os gargalos lógicos de varredura sequencial e ordenação em memória permanecem presentes nas operações complexas e serão o foco da otimização algorítmica.

2. Carga de Trabalho Analisada (Workload)

Para o diagnóstico de desempenho, foram arquitetadas 3 consultas complexas simulando gargalos reais de análise de dados:

1. **Desempenho de Vendas por Categoria:** Múltiplas junções estruturais (`JOIN`) e agregações de totalização.
2. **Ranking de Clientes VIPs:** Uso intenso de funções de janela (`RANK() OVER`) e subconsultas em memória (`CTE`).
3. **Análise de Prejuízo na Produção:** Junções e agregações com cálculos matemáticos aplicados a linhas filtradas.

3. Linha de Base de Desempenho (Baseline)

O comportamento do otimizador de consultas (sem a aplicação de índices adicionais ou reescrita) foi mapeado via `EXPLAIN ANALYZE` , obtendo os seguintes indicadores iniciais:

- **Consulta 1:** Custo total estimado de 0.13. Tempo de execução de 0.322 ms. Estruturas utilizadas: `Nested Loops` , `GroupAggregate` e múltiplas varreduras sequenciais (`Seq Scan`).
- **Consulta 2:** Custo total estimado de 0.11. Tempo de execução de 0.197 ms. Estruturas utilizadas: Agrupamento em hash (`HashAggregate`) e função de agregação de janela (`WindowAgg`) com ordenação na memória (`quicksort`).
- **Consulta 3:** Custo total estimado de 0.07. Tempo de execução de 0.138 ms. Estruturas utilizadas: `HashAggregate` combinado com `Nested Loop` para varredura física sequencial das tabelas de produção.

4. Parte 02: Diagnóstico e Análise de Planos de Execução

Com base nas coletas do `EXPLAIN ANALYZE`, foi identificado três grandes ofensores de desempenho que explicam o custo elevado das consultas atuais:

1. **Gargalo de Filtragem (Seq Scan):**
   - **Diagnóstico:** O SGBD realizou varreduras sequenciais completas nas tabelas (ex: `salesorderheader`), lendo todos os blocos de dados para encontrar registros específicos, resultando em alto custo de Entrada e Saída (I/O) e complexidade de tempo *O(N)*.
   - **Solução Proposta:** Criação de índices B-Tree nas colunas utilizadas nas cláusulas `WHERE` (ex: `status`, `scrappedqty`) para viabilizar o *Index Scan*.
2. **Gargalo de Junção (Nested Loops em cascata):**
   - **Diagnóstico:** A ausência de índices nas chaves estrangeiras forçou o otimizador a utilizar laços aninhados cegos para resolver os `JOINs`. Em consultas com 5 a 6 tabelas transacionais conectadas, isso gera uma degradação severa devido ao produto cartesiano lógico das comparações iterativas.
   - **Solução Proposta:** Indexação das colunas de ligação (Foreign Keys) para permitir que o otimizador escolha algoritmos mais eficientes, como `Hash Join` ou `Merge Join`.
3. **Gargalo de Ordenação e Agregação (Quicksort / HashAggregate):**
   - **Diagnóstico:** As operações de agregação (`SUM`, `GROUP BY`) e as funções de janela (`RANK() OVER`) foram resolvidas em memória (via `quicksort` e agrupamentos de hash). Em cenários de crescimento de volume de dados, isso causará transbordamento de memória para o disco (*temp files*).
   - **Solução Proposta:** Uso estratégico de índices, visto que estruturas em árvore B+ armazenam os dados de forma pré-ordenada, isentando a CPU de alocar espaço de trabalho (`work_mem`) para executar ordenações no momento da consulta.

5. Parte 03: Aplicação das Ações de Tunning

Foram criados 6 índices B-Tree direcionados para resolver os gargalos identificados na análise dos planos de execução:

1. **Índices Parciais (`WHERE status = 5` e `WHERE scrappedqty > 0`):**
   - **Justificativa:** Em vez de indexar tabelas inteiras com milhões de registros, os índices parciais armazenam apenas as linhas que atendem aos filtros das nossas consultas frequentes.
   - **Efeito Colateral Previsto:** Menor consumo de espaço em disco e overhead reduzido em operações de `INSERT` e `UPDATE`, pois o índice só é atualizado quando a linha atende ao predicado do filtro.
2. **Indexação de Chaves Estrangeiras (`customerid`, `productid`, `personid`):**
   - **Justificativa:** A ausência de índices nas FKs impedia o otimizador de utilizar junções eficientes (*Hash Join* e *Index Scan*).
   - **Efeito Colateral Previsto:** Ligeiro aumento no tempo de escrita (`INSERT` / `DELETE`) nessas tabelas transacionais, visto que o SGBD precisa atualizar a estrutura da árvore B+ a cada nova transação, além do custo de armazenamento de blocos adicionais de índice.

6. Parte 04: Comprovação dos Resultados e Limites da Otimização

Após a aplicação das ações de tunning (criação de índices B-Tree compostos, parciais e de chaves estrangeiras), procedeu-se com a re-execução da carga de trabalho via `EXPLAIN ANALYZE` para medição comparativa.

Resultados Obtidos (Pós-Tunning):

- **Consulta 1:** Custo manteve-se em 0.13.
- **Consulta 2:** Custo manteve-se em 0.11.
- **Consulta 3:** Custo manteve-se em 0.07.

Análise Crítica e Limites das Melhorias Obtidas:

Apesar da implementação arquitetural correta dos índices, o otimizador do PostgreSQL optou por não utilizá-los (*Index Scan*), mantendo a estratégia de varredura sequencial (*Seq Scan*). Este comportamento, longe de ser uma falha, demonstra a inteligência do otimizador baseada em custos:

1. **Baixa Cardinalidade Física:** Como a base de dados populada no contêiner atual contém amostragens reduzidas (com estimativas de `rows=1` em tabelas como `salesorderheader` e `workorder`), os dados ocupam um número ínfimo de páginas (blocos) em disco.
2. **Custo de I/O vs. Tamanho:** O SGBD calculou que o custo de acessar a estrutura em árvore do índice e, posteriormente, realizar o *lookup* na tabela original (acesso randômico) seria maior do que simplesmente realizar a leitura sequencial de uma única página de disco na memória RAM.
3. **Fator de Hardware:** Em hardwares de altíssima velocidade (SSD NVMe 2.0), o custo de varredura sequencial em tabelas curtas é virtualmente zero (tempos de execução permaneceram na casa de 0.1 a 0.5 milissegundos).

Conclusão e Recomendações Finais:

O projeto físico foi otimizado com sucesso para escalar. As melhorias algorítmicas projetadas (transformação de *O(N)* para *O(log N)*) estão latentes na estrutura do banco. O limite atual da otimização é exclusivamente o baixo volume de dados da amostra. Recomenda-se que, caso a base receba uma carga massiva de registros em ambiente de produção real, o comando `ANALYZE` seja executado para atualizar as estatísticas do SGBD, momento em que o otimizador passará a priorizar os índices criados para evitar a degradação exponencial de desempenho.